

Reviewer Pack — Minimal Number of Automorphism Groups and Communication Comp...

Entry 5a7ad56b0875 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 17:36:45 UTC

Minimal Number of Automorphism Groups and Communication Complexity Rank Inequality

Entry ID: 5a7ad56b0875 Verdict: INCONCLUSIVE Recorded: 2026-06-03 17:36:45 UTC

1. Conjecture

Field A (mathematical branch): Group Theory (Automorphisms) **Field B** (complexity object): Communication Complexity

Statement:

For every k -CNF φ with n variables, the minimal order of the automorphism group $\text{Aut}(\varphi)$ is upper-bounded by $\Omega(n^2 \log n)$, and this bound is tight for certain CNFs.

Rationale (proposer’s reasoning):

Automorphisms can potentially reduce the communication complexity between two parties solving a problem. By studying the structure of the automorphism group, we may uncover non-trivial connections to the communication complexity. The use of group theory in this context is rarely explored in complexity theory, offering a novel perspective.

Taxonomy category: group_theory_automorphisms (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c8782eb4e8fd99fb

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean order of the automorphism groups across all k -CNFs exceeds $\Omega(n^2 \log n)$ and no seed produces a communication complexity rank $r(\varphi) < |\text{Aut}(\varphi)| / n^2 \log n$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - $\text{aut}(\varphi)$ AND communication complexity - CNF φ AND automorphism group lower bound - group theory automorphisms IN communication complexity

Top relevant hits considered: - [s2:2601.04446] Optimal Depth-Three Circuits for Inner Product
- [s2:2411.00466] Semirigidity and the enumeration of nilpotent semigroups of index three

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_kcnf(n, k):
        cnf = []
        for _ in range(k * n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            if len(set(clause)) == 2:
                cnf.append(clause)
        return cnf

    def is_valid_clause(clause, assignment):
        return any(assignment[abs(lit) - 1] * lit > 0 for lit in clause)

    def evaluate_cnf(cnf, assignment):
        return all(is_valid_clause(clause, assignment) for clause in cnf)

    def find_automorphisms(cnf):
        n = len(cnf)
        automorphisms = []
        for perm in itertools.permutations(range(n)):
            if all(evaluate_cnf(cnf, {perm[i] + 1: val for i, val in enumerate(assignment)})) for a in cnf):
                automorphisms.append(perm)
        return automorphisms

    def communication_complexity_rank(cnf):
        n = len(cnf)
        max_rank = 0
        for perm in itertools.permutations(range(n)):
```

```

        rank = 0
        for clause in cnf:
            if all(perm[abs(lit) - 1] * lit > 0 for lit in clause):
                rank += 1
        max_rank = max(max_rank, rank)
    return max_rank

n_values = [5, 10, 15, 20, 30, 40]
total_metric_value = 0
instances_tested = 0
n_max = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):
        cnf = generate_kcnf(n, k=3)
        automorphisms = find_automorphisms(cnf)
        order = len(automorphisms)
        rank = communication_complexity_rank(cnf)

        if order < n**2 * math.log(n):
            conjecture_holds = False
            counterexample = f"n={n}, |Aut( $\phi$ )|={order}, r( $\phi$ )={rank}"

        total_metric_value += order
        instances_tested += 1
        n_max = max(n_max, n)

    return {
        "metric_name": "Order of Automorphism Group",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **{result}}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")

```

```

else:
    print("RESULT: INCONCLUSIVE no seeds tested")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

eback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 91, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 64, in run_trial
    automorphisms = find_automorphisms(cnf)
                    ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 39, in find_automorp
    if all(evaluate_cnf(cnf, {perm[i] + 1: val for i, val in enumerate(assignment)})) for assignment in i
1, 1], repeat=n)):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 39, in <genexpr>
    if all(evaluate_cnf(cnf, {perm[i] + 1: val for i, val in enumerate(assignment)})) for assignment in i
1, 1], repeat=n)):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 33, in evaluate_cnf
    return all(is_valid_clause(clause, assignment) for clause in cnf)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 33, in <genexpr>
    return all(is_valid_clause(clause, assignment) for clause in cnf)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 30, in is_valid_clau
    return any(assignment[abs(lit) - 1] * lit > 0 for lit in clause)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_028e2c4e.py", line 30, in <genexpr>
    return any(assignment[abs(lit) - 1] * lit > 0 for lit in clause)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyError: 0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means it did not complete its execution to provide evidence for the conjecture. | next: Re-run the test with appropriate error handling and verification of results to determine if the conjecture is supported or falsified.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14380
2	propose	ollama_remote	glm4:latest	0	0	18255
3	preregistration	ollama_remote	glm4:latest	0	0	10010
4	novelty	ollama_remote	glm4:latest	0	0	8336
5	novelty	ollama_remote	glm4:latest	0	0	13455
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14232
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9779
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	78089
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18634
10	judge	ollama_remote	glm4:latest	0	0	10803

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 195975 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5a7ad56b0875.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5a7ad56b0875.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5a7ad56b0875.tar.gz` (if generated)